

CS570: Big Data Processing & Analytics

Week 1 Lab — First Contact with Spark (Student Handout)

What This Lab Is About

In this lab, you will get your first hands-on experience with Apache Spark using Jupyter Lab. The goal is *not* to optimize performance or write production code. The goal is to understand how Spark behaves as a system.

You will:

- Run a small number of Spark commands
 - Observe when Spark actually does work
 - Connect what you see to concepts discussed in lecture
-

Before You Begin

Required Setup

You will need:

- Python 3.11
- Java 17 (required by Spark)
- Jupyter Lab
- PySpark installed

Important: Python 3.13 is currently *too new* for Spark/PySpark. If you are on Python 3.13, you will need a separate environment with Python 3.11 for this lab.

If your environment does not work immediately, pair up with a classmate and continue. Understanding the behavior is more important than finishing every step alone.

Installation Notes (Read Once Before Class)

Why Java Is Required

Apache Spark runs on the JVM. Even when you use PySpark, Python is only the front-end API. The actual execution engine is written in Scala/Java, which is why Java is mandatory.

One-Time Setup Checklist

1. Install Java 17

- macOS (Homebrew): `brew install temurin@17`
- Windows: install *Eclipse Temurin 17 (LTS)*
- Linux: install an OpenJDK 17 package

2. Verify Java installation

```
java -version
```

You should see version 17.x

3. If Python 3.11 is not installed yet

First check which Python versions you have:

```
python --version
```

```
python3 --version
```

If you do not have Python 3.10 or 3.11, install one of them:

macOS (Homebrew):

```
brew install python@3.11
```

Windows: Download Python 3.11 from python.org and check “Add Python to PATH” during installation.

Linux: Install a `python3.11` package using your system package manager.

After installation, verify:

```
python3.11 --version
```

4. Create a Python virtual environment

```
python3.11 -m venv cs570
```

```
source cs570/bin/activate # macOS/Linux
```

```
cs570\Scripts\activate # Windows
```

5. Install required Python packages

```
pip install pyspark jupyterlab pandas
```

6. Quick Spark sanity test

You can run this either:

- inside a Python shell (after activating the virtual environment), **or**
- inside a Jupyter Lab cell

```
from pyspark.sql import SparkSession
spark =
SparkSession.builder.master("local[*]").appName("cs570-week1").ge
tOrCreate()
spark.range(10).count()
```

If this returns **10**, Spark is working.

If you get stuck on setup, come to class anyway. You can pair up and still complete the lab.

Lab Structure (What You Will Do)

1. Launch JupyterLab and open the provided notebook
2. Verify Spark runs inside the notebook
3. Load a dataset into Spark
4. Define transformations (no execution yet)
5. Trigger execution using an action
6. Observe Spark's behavior and reflect

Step 1: Start JupyterLab and Open the Notebook

1. Make sure your virtual environment is activated
2. Start JupyterLab:
`jupyter lab`
3. Your browser should open automatically
4. Navigate to and open the provided Week 1 Spark notebook
`week1_spark_intro.ipynb`
5. Run the first cell to import libraries and create a `SparkSession`

You should see Spark start successfully without errors.

Kernel choice: When you open the notebook, select the kernel named: "Python 3.11 (cs570)" or "cs570". Do not use the system Python 3.13 kernel!

Common issue (and fix): If you see an error like `ModuleNotFoundError: No module named 'pyspark'`, the notebook is using the wrong kernel. Make sure you selected Python 3.11 (cs570) and that JupyterLab was started from an activated `cs570` virtual environment.

If you cannot find such a kernel, install and register an explicit kernel for this venv (one-time):

```
pip install ipykernel
python -m ipykernel install --user --name cs570 --display-name "Python 3.11 (cs570)"
```

Step 2: Load Data

The dataset used in this lab is a small synthetic clickstream-style dataset (simplified website event logs). It is generated automatically inside the notebook so we can focus on Spark's behavior rather than data cleaning or downloads.

Run the cell that loads the dataset using `spark.read.csv(...)`.

Then:

- Display a few rows of data
- Inspect the inferred schema

Checkpoint: Has Spark done any heavy computation yet?

Step 3: Define Transformations

Run the cells that:

- Select a subset of columns
- Filter rows based on a simple condition
- Chain multiple transformations together

Do **not** run any actions yet.

Question: Why do these steps run instantly?

Step 4: Trigger Execution

Now run a cell that calls an action, such as:

- `count()`
- `collect()`

Observe what happens:

- Does execution take longer?
 - Do you see activity in the Spark UI? (typically available at <http://localhost:4040>)
-

Step 5: Observe and Reflect

If available, open the Spark UI and look at:

- Jobs
- Stages
- Tasks

Reflection questions:

- When did Spark actually execute work?
 - What caused execution to start?
 - How does this relate to lazy evaluation?
-

Troubleshooting (If Spark is “not found” in Jupyter)

Symptom A: `ModuleNotFoundError: No module named 'pyspark'`

This means the notebook is using the wrong Python environment.

Fix

1. Close JupyterLab
2. In terminal, activate the venv:
`source cs570/bin/activate`
3. Confirm PySpark is installed in that venv:
`python -c "import pyspark; print(pyspark.__version__)"`

4. Start JupyterLab from the activated venv:
`jupyter lab`
5. In the notebook: Kernel → Change Kernel → Python 3.11 (cs570)

Symptom B: Java / Spark startup error

If `pyspark` imports but `SparkSession` fails to start, verify Java:

```
java -version
```

You should see Java 17.

Wrap-Up

Before leaving, be ready to discuss:

- One thing that surprised you
- One question you still have about Spark

This lab is intentionally exploratory. Confusion is normal at this stage.

Next week, we will look under the hood at Hadoop storage and execution, which Spark builds on.